Environmental Clearance Pre-Screening Service

A multi-agent system for screening projects under the EIA Notification, 2006

Mini Project Report

UE24AM342AA5: ML System Design and AgentOps

Mini Project 13, Jackfruit (Level 3)

Submitted by:

Ashwin C H

Pallavi B

Dhanush S R

Department of Computer Science and Engineering (AI and ML)

PES University

November 2026

Contents

1. Introduction

2. Problem Statement

3. Corpus and Data Preparation

4. Agent Design

5. Architecture and Deployment

6. Resilience and Guardrails

7. Evaluation

8. Performance, Cost and Caching

9. Problems We Ran Into

10. Limitations and Future Work

11. Conclusion

References

1. Introduction

Before a new project such as a cement plant, a mine or a thermal power station can be built in India, it may need prior environmental clearance under the Environmental Impact Assessment (EIA) Notification, 2006. Whether a project needs clearance, and which authority has to give it, is decided at an early stage called screening. Screening answers two questions: is the activity listed in the Schedule of the notification at all, and if it is, does it fall under Category A (appraised by the Central Government through MoEFCC) or Category B (appraised by the State Environment Impact Assessment Authority).

Getting this wrong is expensive. A clearance granted at the wrong level can be set aside later, sometimes after construction has already started. At the same time, the rules are spread across one principal notification and a long list of amendments and office memoranda, so working out the category by hand is slow and easy to get wrong.

For this mini project we built a service that takes a project description and returns a screening opinion: no clearance needed, Category B (State), or Category A (Central), along with the exact provisions it relied on and verbatim quotes from them. When the evidence does not settle the question, the service escalates the case to a human instead of guessing. It is important to say that the system never makes the clearance decision. It produces an opinion with evidence, and an accredited consultant decides.

The goal of the course project was a production style service rather than a notebook, so a large part of the work went into the parts around the model: a data contract, a queue, separate agent services, verification, resilience when the LLM provider fails, observability, load testing against a declared SLO, and a runbook. This report describes what we built, the decisions we made, how we evaluated it, and what did not work.

2. Problem Statement

2.1 Scope

The service accepts a project description with its sector, capacity, location and distances to sensitive features such as national parks. It validates the input against a published contract, decides whether the Schedule applies and which category the project falls under, and routes it with a cited justification. Impact assessment itself, public hearings and the actual appraisal are out of scope.

2.2 The hard case: the General Condition

The Schedule of the notification is a table of activities and thresholds. If you only read the table, most cases look simple. The difficulty is a note printed after the table, called the General Condition. It says that a project which would normally be Category B must be treated as Category A if it is located within 5 km of a protected area, a critically polluted area, an eco-sensitive area, or an inter-State or international boundary (10 km for a few

specific items). This condition is not repeated in any row of the Schedule.

The reference case from the problem statement shows why this matters. An integrated cement plant of 0.8 million tonnes per annum is Category B according to Schedule item 3(b). But if the same plant sits 2 km from Gir National Park, the General Condition moves it to Category A. We checked this against our own index: for the query describing this plant, the top six semantic search results contain row 3(b) but do not contain the General Condition. A simple retrieval augmented generation (RAG) pipeline that searches once and answers would therefore give the wrong category with full confidence.

This one case shaped most of our design. The system has to notice that a row refers to another provision, go and fetch that provision, and then someone independent has to check that it was actually applied.

3. Corpus and Data Preparation

All documents were downloaded from the MoEFCC website and the PARIVESH portal. The manifest is kept in data/corpus_manifest.csv.

Document type	**Count**
Consolidated notification (amended up to March 2025)	1
Principal notification, 2006	1
Gazette amendments (S.O. notifications)	77
Office memoranda	44
Total	123

Table 1: Documents in the corpus.

Out of the 123 documents, 71 have a text layer and were indexed, giving 1,380 chunks. 51 documents are scanned images. OCR was out of scope for us, so these are recorded in the manifest as skipped rather than silently ignored. One file was a corrupt download that the PDF reader could not open.

3.1 Chunking

Our first attempt used fixed windows of about 500 characters. This split Schedule rows in the middle and in some places cut the General Condition away from its 10 km proviso, which is exactly the text we cared about most. We replaced it with structure aware chunking:

- one chunk per Schedule row, with an id such as schedule:3(b);
- the General Condition and the Specific Condition each as their own chunk;
- paragraphs of the notification split by their roman numbered sub-paragraphs;
- footnotes (which record amendments) kept as separate chunks.

While chunking we also record cross references found in the text itself. When a row contains the phrase "General Condition shall apply", that link is stored with the chunk. Footnotes are linked to the rows they amend, together with whether the amendment is still in force. These links are what the find_modifiers tool follows later.

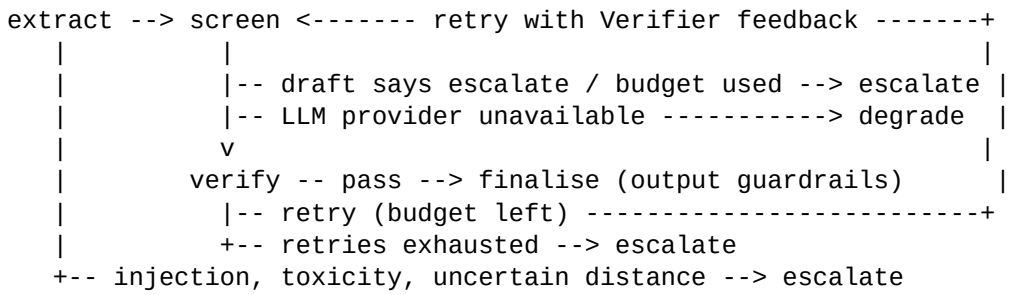
3.2 Index

Chunks are embedded with the all-MiniLM-L6-v2 sentence transformer running locally and stored in a FAISS index through LangChain. The indexer is incremental: it only re-embeds chunks whose text changed, and it keeps an embedding cache on disk. We chose local embeddings because embedding through a paid API was the only real cost item in the problem statement, and a local model removed it completely.

4. Agent Design

4.1 Topology

The run is a LangGraph graph. We picked LangGraph over CrewAI because we wanted an explicit graph with a conditional retry edge and hard limits, and LangGraph gave us more control over that loop. The flow is:



The components are described in Table 2. Only the Screener and the Verifier are LLM agents. They have separate prompts, separate tool sets and run in separate containers.

Component	Role	Tools
Extractor (deterministic)	Runs input guardrails, pulls out sector, capacity and distances, and escalates injection attempts or uncertain distances before any model call.	none
Screener (LLM agent)	Finds the Schedule row, applies thresholds, looks for provisions that modify the row, and drafts a cited opinion or decides to escalate.	search_corpus, get_clause, find_modifiers, get_request_fields
Verifier (LLM agent, own service)	Checks the draft against the text that was actually retrieved; sends it back with feedback or escalates.	get_clause, search_corpus
Rule engine	Used only when no model is reachable. Produces a provisional opinion that is attached to an escalation, never returned as an answer.	none

Table 2: Components of a run.

The hand off between the two agents is a typed object, DraftDetermination, which holds the category, the route, the Schedule item, the basis, the citations with verbatim quotes, and the list of modifiers that were checked. All passages retrieved during the run are passed along with it so the Verifier can check citations against them.

4.2 Retrieval as a tool

We did not hard code the retrieval order. The Screener decides whether to search, what to search for and whether to follow a cross reference. For the Gir case a typical run looks like this: it calls search_corpus with the project type and finds schedule:3(b); it reads the row with get_clause and sees "General Condition shall apply"; it calls find_modifiers on schedule:3(b), which returns the General Condition, whether the item is in the 10 km proviso, amendment footnotes and later notifications that mention the item; and it then drafts Category A citing 3(b), the General Condition and paragraph 4(ii).

We considered two other ways to reach the General Condition. Hard coding "cement leads to the General Condition" would have passed the reference case and failed for every other row. Running a second similarity search does not help either, because the wording of the General Condition does not look like the project description. Following links that were extracted from the text at ingestion was the only approach that generalised.

4.3 Verification

The Verifier runs deterministic checks first, because they are cheap and cannot be argued with, and then an LLM review:

- every citation must resolve to a passage that was retrieved in this run;
- every quote must actually appear in its passage, which catches invented evidence;

- category and route must agree, and a Schedule row (or paragraph 2 for no clearance) must be cited;
- the modifier check: if the cited row says the General Condition applies, the General Condition must have been retrieved, and if the project is within the distance, a Category B draft is rejected;
- a row whose amendment has been quashed or is not in force leads to escalation;
- finally a second model reads the full text of every cited passage and reports unsupported claims or ignored provisions.

If a check fails, the draft goes back to the Screener with specific feedback, for example telling it that item 3(b) says the General Condition applies and that it should call find_modifiers. This happens at most twice. After that the run escalates. We chose a separate Verifier over a self check because a model reviewing its own draft in the same conversation tends to agree with itself.

4.4 Loop limits

Every run has hard limits: at most 24 steps (a normal run uses 8 to 12), at most 12 tool calls (normally 3 to 5), a wall clock deadline of 90 seconds, and at most 2 verification retries. Each LLM call and each tool call also has its own timeout. Hitting any of these limits escalates the run with a reason code such as LIMIT_STEPS or LIMIT_WALL_CLOCK.

4.5 Prompts

Prompts are stored as versioned files under agent/prompts/v1 and agent/prompts/v2. The version and a hash of the prompt are recorded on every model call and in every result, so any answer can be traced back to the exact prompt that produced it. Version 1 is a short baseline. Version 2 spells out the procedure step by step: find the row, apply the threshold, call find_modifiers, and search again based on what was read.

5. Architecture and Deployment

The whole system runs with a single docker compose up command and does not need an API key, because by default an OpenAI compatible stub service answers in place of the real model. Switching to a real provider such as NVIDIA NIM is only a change in the .env file. No credentials, endpoints or model names are in the code.

Service	Why it is separate
api	FastAPI. It is I/O bound and must keep answering polls while agents are running. Validates the contract, quarantines bad input, applies rate limits.
agent-worker (x N)	Runs the Extractor and Screener. It is the heavy part, so it scales on queue depth.
verifier-agent	The second agent. It can fail on its own (the worker then escalates instead of releasing an unverified answer) and can be released on its own.
indexer	One shot batch job with a different resource profile. Everything else waits until it has built the index.
redis	The only shared state: queue, run store, events, caches, rate limits.
llm-stub	Stand-in for the provider, used for load tests.
prometheus, grafana, jaeger	Metrics, the SLO dashboard and distributed traces, kept apart from the application.

Table 3: Services under Docker Compose.

Execution is asynchronous. A POST to /v1/screenings returns HTTP 202 with a run id, the run is put on a Redis queue, and the caller polls. We chose polling over callbacks because our callers (the browser client, curl and Locust) do not have a public endpoint to receive a callback. The queue also means that a burst of requests waits in line instead of overloading the LLM provider.

Every run can be reconstructed through GET /v1/screenings/{id}/trace, which returns each tool call, the query used, the passage ids returned, the prompt version, per step latency, tokens, cost and the verification outcome. Logs are JSON with a correlation id and run id on every line.

6. Resilience and Guardrails

6.1 LLM gateway

All model calls go through agent/llm/gateway.py. For each model on the path from the primary model to the fallback model, the call passes through a circuit breaker, then retry with exponential backoff (0.5 s, 1 s, 2 s with jitter, only for transient errors), then a timeout. If every model has failed or its breaker is open, the run takes the degraded path: the rule engine writes a provisional opinion and the run is escalated with that opinion attached. A run that did not complete normally is never returned as a confident answer.

We can demonstrate this with POST /admin/chaos, which makes every model call fail with HTTP 503. On our local stack the first run made one call and three retries, took 3.7 seconds and degraded. On the next run the fifth consecutive failure had opened the breaker, so the run failed fast in 0.7 seconds.

If the Verifier service is unreachable, the run is escalated as VERIFIER_UNAVAILABLE. If a worker process dies in the middle of a run, its message stays in a processing list in Redis, and a reaper in another worker re-queues it once before marking it as WORKER_LOST.

6.2 Guardrails

On input we check for prompt injection and toxicity, and the project description is passed to the model inside clear data delimiters. All tools are read only. On output, a filter runs before anything is returned. Two of our evaluation scenarios are injection attempts, and both are escalated by the Extractor before any model is called.

7. Evaluation

7.1 Scenario set and metrics

We wrote 28 scenarios in eval/scenarios.json. Each one has the expected outcome, the passages that support it and a short reason. The set includes the Gir reference case, distances on either side of the boundary (4.9 km and 5.1 km), the 10 km proviso, all four General Condition triggers, quashed amendments, missing and estimated distances, an out of corpus case, two injection attempts, no clearance cases, and the problem statement's own wording taken literally. Every scenario is validated against the published contract first.

The metrics are defined in eval/metrics.py. We computed them ourselves rather than through TruLens, using equivalent definitions:

- **Groundedness:** share of citations that resolve to retrieved text and whose quote is verbatim.
- **Context relevance:** share of retrieved passages that were reference passages or were cited.
- **Context recall:** share of reference passages that were retrieved. For the Gir case this answers whether the General Condition was reached.
- **Faithfulness:** the final draft passes the Verifier's deterministic checks when they are re-run independently.
- **Hallucination rate:** share of runs with at least one citation that does not resolve or whose quote is not in the passage.

7.2 Deliberate change: removing find_modifiers

To see whether find_modifiers was actually pulling its weight, we ran the evaluation with that tool removed, so the Screener only had plain similarity search. Our hypothesis was that the General Condition would no longer be reached on the first attempt, the Verifier would reject Category B drafts, and recovering would cost retries and latency. These numbers were measured with the stub model, which behaves like a naive single pass RAG model

when it has no modifier tool and follows the Verifier's feedback on a retry.

Metric	Before	After (no find_modifiers)	Change
Route accuracy	1.00	1.00	+0.00
Groundedness	1.00	1.00	+0.00
Context relevance	0.18	0.30	+0.12
Context recall	1.00	0.98	-0.02
Faithfulness	1.00	1.00	+0.00
Hallucination rate	0.00	0.00	+0.00
Runs rejected by the Verifier at least once	0%	68%	+68 points
LLM calls per run	4.1	6.8	+63%
Cost per 100 runs	$0.175	$0.261	+49%

Table 4: Effect of removing find_modifiers (stub model, 28 scenarios).

Without the modifier tool, about two drafts in three ignored the General Condition and were caught by the Verifier, which named the missing provision. The retry then recovered the right answer, so accuracy did not drop, but it took 63% more model calls. In other words, removing the step that looks for modifying provisions was paid for by the verification step. Context relevance went up, because fewer loosely related passages were pulled in, which tells us find_modifiers returns more than it needs to.

7.3 Run with live models

For most of the project our NVIDIA API key was rejected (see INC-006 in Section 9), so the full evaluation was developed against the stub. After getting a working key we ran six key scenarios with prompt v1 on NVIDIA NIM, using mistralai/mistral-large-4-1t-a52b as the primary model and nvidia/nemotron-3.5-lightning-30b-a3b as the fallback.

Case	Scenario	Expected	Result
RC-1	Cement plant 2 km from Gir National Park	Category A	Category A (correct)
RC-2	Same plant, 7 km away	Category B	Category B (correct)
RC-5	Thermal plant 8 km away (10 km proviso)	Category A	Category A (correct)
RC-26	Generic "industrial project"	Escalate (sector ambiguous)	Escalate (correct)
RC-9	Beach resort under CRZ (out of corpus)	Escalate (out of corpus)	Escalate (insufficient information)
RC-24	Solar PV park, not a listed activity	No clearance	Escalate (wall clock limit)

Table 5: Results on live models.

Four of six routes were correct. Groundedness and faithfulness were both 1.00, the hallucination rate was 0, and there were no unsafe answers. The two misses were both escalations, so the service handed the case to a human instead of giving a wrong answer. RC-9 escalated for a slightly different reason than expected, and RC-24 ran out of time. Importantly, all three General Condition cases were correct, including the 10 km proviso. The p50 latency was 49 seconds and the p95 was 108 seconds, which is above our SLO; this is discussed in the next section.

8. Performance, Cost and Caching

8.1 SLO

We wrote down the SLO in docs/SLO.md and committed it before running any load test, so the target could not be chosen after seeing the results. The objective is a p95 latency of at most 60 seconds for a complete run, measured from submission until the result is available, at 10 concurrent users, with the stub model at 1.5 seconds per call. Our reasoning was that a slow answer costs far less than a wrong one, so the design deliberately spends time on correctness, but a consultant at a web form will not wait much more than a minute.

An early mistake here was timing only the POST request, which measures about 10 ms of queueing and nothing else. The Locust file now reports each complete run as a custom request called full_run.

8.2 Load test results

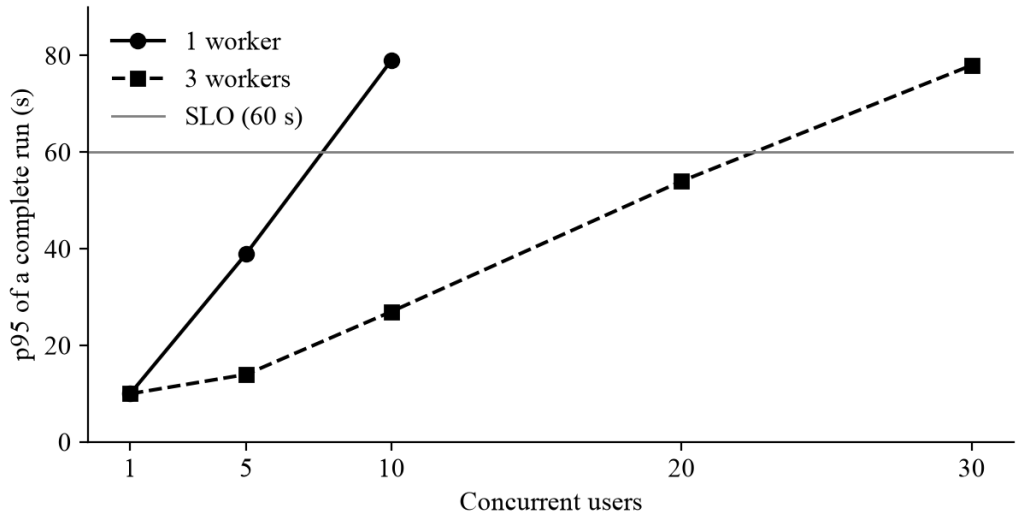


Figure 1: p95 latency of a complete run against concurrent users.

Users	1 worker: runs/s	p95	3 workers: runs/s	p95
1	0.09	10.0 s	0.09	10.0 s
5	0.11	39 s	0.37	14 s
10	0.13	79 s (missed)	0.36	27 s
20			0.36	54 s
30			0.36	78 s (missed)

Table 6: Load test on the local multi-process stack, 90 s per level.

With one worker the SLO fails at 10 users. With three workers it holds up to 20 users and fails at 30. Going from one to three workers multiplied throughput by 3.2.

The bottleneck is worker concurrency. A worker handles one run at a time, and a run takes about 8 seconds, almost all of it waiting on five model calls of 1.5 seconds each. So throughput is capped at roughly the number of workers divided by 8.3 seconds, which is 0.12 runs per second for one worker and 0.36 for three, exactly what we measured. Above that cap every extra user only adds time in the queue. Since scaling was linear, no shared component such as the API, Redis or the Verifier was saturated. Because the work is mostly waiting, the better next step would be running several runs concurrently inside each worker rather than adding more containers.

8.3 Cost

Tokens and cost are recorded at call time for every step and every agent, and exported to Prometheus. With the stub, a run uses about 7,960 tokens, which comes to about $0.18 per 100 runs at the configured reference prices. On the live models the cost was about $0.85 per 100 runs at about 35,000 tokens per run. The largest cost driver is the Screener's multi-turn context, at about 8,200 prompt tokens per run against about 1,500 for the Verifier, because every turn re-sends the conversation including earlier tool results.

8.4 Caching

We cache at three levels: the whole run (keyed on the payload, prompt version, index version and models, and only for completed, non-degraded runs), retrieval results (keyed on the query and index version), and embeddings on disk.

Setting	Runs/s	p50	p95	Cost per run
Cache off (unique payloads)	0.37	22 s	27 s	full
Cache on (repeated payloads)	2.62	0.01 s	9.1 s	about 80% lower

Table 7: Effect of the run cache at 10 users and 3 workers.

With an 80% hit rate the cost per run dropped by about 80%. During this test the per-client rate limit (20 submissions per minute) rejected 64 submissions, because instant answers let each simulated user submit very quickly. That is the limiter working as a cost control.

9. Problems We Ran Into

We kept an incident log in docs/RUNBOOK.md for every real failure we hit while building and testing. There are fourteen entries. A few of them taught us the most:

- **INC-003, every run stuck in QUEUED.** Under Docker Compose no request ever finished. Run records were stored in a Python dictionary inside the API process, so the worker container took the run id off the queue, found nothing in its own memory and returned silently. We moved the run store, events, caches and feedback to Redis.
- **INC-004, worker never processed jobs.** The Redis client's socket timeout was 0.2 seconds, shorter than the 2 second blocking pop, so every pop raised a timeout. We raised the timeout and switched to BLMOVE into a processing list with a reaper.
- **INC-005, a load test that measured nothing.** Locust was sending a field the contract forbids, so 100% of submissions were rejected with 422 and looked very fast. The payload now follows the published contract.
- **INC-006, NVIDIA key rejected.** Every call returned HTTP 403. Worse, the old client swallowed such errors and returned fake output. We replaced it with a client that raises errors so that the breaker and fallback path can see them.
- **INC-009, retries that never stopped at two.** We were incrementing the retry counter inside a LangGraph routing function, and LangGraph discards state changes made by routers. Counters are now written only by nodes. After the fix, the retry rate in the evaluation went from a false 0% to the real 68%.
- **INC-011, correct answers rejected.** Real models tidy up quotes from table rows by adding separators or changing case, so the exact substring check failed. The check now accepts ellipses and separators if 85% of the quote matches the passage verbatim, while invented quotes still fail.
- **INC-013, a run that took 263 seconds.** A solar case ran far past the 90 second deadline because retries ignored the deadline. Each call's timeout is now capped at the time left in the run, and the same case now answers in 18 seconds.

The runbook also documents two failure modes in detail, an unavailable LLM provider and a growing queue, each with symptoms, how to confirm it and how to recover.

10. Limitations and Future Work

- 51 scanned documents are not indexed because we did not do OCR. Cases that depend only on them would escalate. Adding OCR is the obvious next step.
- Context relevance is low at 0.18 because find_modifiers is generous. A possible fix is to rank later notifications by similarity to the row and keep only the top one.

- The live model p95 of 108 seconds is above the 60 second SLO. Running several runs concurrently inside each worker should help, since most of the time is spent waiting on the model.
- The live evaluation only covers six scenarios. The full 28 scenario set and the v1 versus v2 prompt comparison still need to be run on real models.
- The stub Verifier model always agrees, so the LLM review half of verification is only really exercised with a real model. The deterministic half is covered by tests.
- Circuit breakers are per process, so each worker learns about an outage separately. A shared breaker in Redis was not built.
- cAdvisor is optional because Docker Desktop on macOS can refuse its host mounts.

11. Conclusion

We built a multi-agent pre-screening service for the EIA Notification, 2006 that runs as nine services under Docker Compose. The main idea is that a correct category matters more than a fast one. The Screener can follow the cross reference from a Schedule row to the General Condition using a tool driven by links extracted at ingestion, an independent Verifier refuses drafts that ignore it, and the service escalates whenever the evidence does not settle the question.

The evaluation supports this design. Removing the modifier tool did not reduce accuracy only because the Verifier caught the mistakes, at the cost of 63% more model calls. On live models, every General Condition case was answered correctly and both misses were safe escalations. The biggest open problems are latency on real models, the unindexed scanned documents and a larger live evaluation. Building the parts around the model, such as the queue, the data contract, the traces and the incident log, took more of our time than the agents themselves, and that was probably the most useful lesson of the project.

References

[1] Ministry of Environment, Forest and Climate Change, Government of India. Environmental Impact Assessment Notification, S.O. 1533(E), 14 September 2006, consolidated up to March 2025.

[2] Ministry of Environment, Forest and Climate Change. Amendments and office memoranda to the EIA Notification, 2006. PARIVESH portal, parivesh.nic.in.

[3] LangGraph documentation. LangChain, Inc.

[4] J. Johnson, M. Douze and H. Jegou. Billion-scale similarity search with GPUs (FAISS). IEEE Transactions on Big Data, 2019.

[5] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

[6] Course material, UE24AM342AA5 ML System Design and AgentOps, Mini Project 13 problem statement, PES University.